

ZOS – Semestrální práce

Souborový Systém s využitím PseudoFAT
systému

Vypracoval: Martin Schön
Studijní číslo: A22B0144P
Datum: 31. října 2024

Obsah

1	Zadání	3
1.1	Obecné zadání	3
1.2	Rozšíření zadání	5
2	Struktura souborového systému	6
2.1	Informace o souborovém systému	6
2.2	FAT	6
2.3	Datová část	6
3	Implementace	7
3.1	Předměty a FAT	7
3.2	Utils	8
4	Příkazy	8
4.1	Mapování příkazů	8
4.2	Příkaz cp	8
4.3	Příkaz mv	8
4.4	Příkaz rm	9
4.5	Příkaz mkdir	9
4.6	Příkaz rmdir	9
4.7	Příkaz ls	9
4.8	Příkaz cat	9
4.9	Příkaz cd	10
4.10	Příkaz pwd	10
4.11	Příkaz info	10
4.12	Příkaz incp	10
4.13	Příkaz outcp	10
4.14	Příkaz load	10
4.15	Příkaz format	11
4.16	Příkaz check	11
4.17	Příkaz bug	11
4.18	Příkaz help	11
4.19	Příkaz exit	11
5	Uživatelská příručka	12

1 Zadání

1.1 Obecné zadání

Tématem semestrální práce bude práce se zjednodušeným souborovým systémem založeným na pseudoFAT. Vaším cílem bude splnit několik vybraných úloh. Základní funkčnost, kterou musí program splňovat. Formát výpisů je závazný. Program bude mít jeden parametr a tím bude název Vašeho souborového systému. Po spuštění bude program čekat na zadání jednotlivých příkazů s minimální funkčností viz níže (všechny soubory mohou být zadány jak absolutní, tak relativní cestou):

- Zkopíruje soubor s1 do umístění s2

```
cp s1 s2
```

s1 - zdrojový soubor

s2 - cílový soubor

- Přesune soubor s1 do umístění s2, nebo přejmenuje s1 na s2

```
mv s1 s2
```

s1 - zdrojový soubor

s2 - cílový soubor

- Smaže soubor s1

```
rm s1
```

s1 - soubor, který se má smazat

- Vytvoří adresář a1

```
mkdir a1
```

a1 - adresář, který se má vytvořit

- Smaže prázdný adresář a1

```
rmdir a1
```

a1 - adresář, který se má smazat

- Vypíše obsah adresáře a1, bez parametru vypíše obsah aktuálního adresáře

```
ls a1
```

a1 - adresář, jehož obsah se má vypsát

- `ls` - vypíše obsah aktuálního adresáře
- Vypíše obsah souboru `s1`

`cat s1`

`s1` - soubor, jehož obsah se má vypsát
 - Změní aktuální cestu do adresáře `a1`

`cd a1`

`a1` - adresář, do kterého se má změnit aktuální cesta
 - Vypíše aktuální cestu

`pwd`
 - Vypíše informace o souboru/adresáři `s1/a1` (v jakých clusterech se nachází)

`info a1/s1`

`a1/s1` - soubor/adresář, o kterém se mají vypsát informace
 - Nahraje soubor `s1` z pevného disku do umístění `s2` ve vašem FS

`incp s1 s2`

`s1` - zdrojový soubor

`s2` - cílový soubor
 - Nahraje soubor `s1` z vašeho FS do umístění `s2` na pevném disku

`outcp s1 s2`

`s1` - zdrojový soubor

`s2` - cílový soubor
 - Načte soubor z pevného disku, ve kterém budou jednotlivé příkazy, a začne je sekvenčně vykonávat. Formát je 1 příkaz/lířádek

`load s1`

`s1` - soubor, ve kterém jsou příkazy
 - Příkaz provede formát souboru, který byl zadán jako parametr při spuštění programu na souborový systém dané velikosti. Pokud už soubor nějaká data obsahoval, budou přemazána. Pokud soubor neexistoval, bude vytvořen

`format <size>MB`

`size` - velikost souborového systému v MB

Budeme předpokládat korektní zadání syntaxe příkazů, nikoliv však sémantiky (tj. např. `cp s1` zadáno nebude, ale může být zadáno `cat s1`, kde `s1` neexistuje).

1.2 Rozšíření zadání

- Maximální délka názvu souboru bude $8+3=11$ znaků (jméno.přípona) + `\0` (ukončovací znak v C/C++), tedy 12 bytů.
- Každý název bude zabírat právě 12 bytů (do délky 12 bytů doplníte `\0` - při kratších názvech).

Nad vytvořeným a naplněným souborovým systémem umožněte provedení následujících operací:

- Kontrola konzistence souborového systému – pokud login studenta začíná j-r

příkaz `check` – Zkontroluje, zda je souborový systém nepoškozen.

příkaz `bug s1` – Poškodí souborový systém, dle vašeho uvážení, tak aby bylo možné ověřit funkcionalitu příkazu `check`.

2 Struktura souborového systému

Souborový systém ve formátu <name>.dat je rozdělen na několik částí. První část obsahuje informace o souborovém systému, následuje tabulka clusterů (FAT), kořenový adresář a samotná data.

2.1 Informace o souborovém systému

Obsahem této části je informace o souborovém systému: Tyto informace jsou:

- Signature – identifikace souborového systému
- Velikost souborového systému – v MB ($1\text{MB} = 1024\text{KB} = 1024 \cdot 1024\text{B}$)
- Velikost clusteru – v B ($1\text{ cluster} = 1\text{ sektor}$)
- Počet clusterů – celkový počet clusterů
- Adresa FAT – adresa FAT tabulky
- Adresa kořenového adresáře – adresa kořenového adresáře

2.2 FAT

FAT tabulka obsahuje informace o jednotlivých clusterech. Každý cluster má svůj záznam v tabulce, který obsahuje stav clusteru. Stav clusteru může být:

- Volný – cluster je volný
- Obsazený – cluster je obsazený
- Řetězec – cluster je součástí řetězce clusterů
- Poškozený – cluster je poškozený

2.3 Datová část

Kořenový adresář obsahuje informace o jednotlivých složkách a adresářích. Samotný adresář se nachází na začátku datové části souborového systému neboli v nultém clusteru. Pokud je cluster přiřazen složce ukládá si informace o dalších složkách nebo souborech, které dále odkazují na další clustery. Pokud je cluster přiřazen souboru ukládá si informace o souboru.

3 Implementace

Program byl implementován v jazyce C++ verze 23. Na začátku programu se zkontroluje počet argumentů a formát souboru, který reprezentuje souborový systém. Následně se načtou informace o souborovém systému, pokud již existoval, nebo se vytvoří nový souborový systém. Nový, či poškozený souborový systém bude první vyžadovat formátování na počáteční velikost. Pokud byl souborový systém načten, program bude čekat na zadání příkazů.

Konzole při spuštění systémů vždy zobrazuje aktuální cestu, kde se nachází. Dále se čeká na zadání příkazu, který určuje, co se má provést. Jednotlivé příkazy se rozdělují podle počtů argumentů:

- `command` – příkaz bez argumentů
- `command arg1` – příkaz s jedním argumentem
- `command arg1 arg2` – příkaz s dvěma argumenty

Pokud nastane stav, kdy se souborový systém poškodí bude možné zadat pouze příkazy `exit`, `help`, `check` a `load`.

3.1 Předměty a FAT

Popis souborového systému je reprezentován třídou `Description`. Ta obsahuje informace o souborovém systému. Každý soubor a adresář je reprezentován třídou `DirectoryItem`. Ta obsahuje informace o souboru nebo adresáři, jako je jméno, jestli se jedná o soubor nebo o adresář, velikost, pozice ve FAT a rodičovský adresář.

Dále jsou nadefinovány konstanty:

- `FAT_UNUSED` – `INT32_MAX - 1`
- `FAT_FILE_END` – `INT32_MAX - 2`
- `FAT_BAD_CLUSTER` – `INT32_MAX - 3`
- `FORMAT_CLUSTER_SIZE` = 1024

FAT tabulka je reprezentována polem `Clusterů`, kde na každém indexu je uložen jeden `int32_t`. Velikost pole je `Clusterů` je:

$$PocetClusteru = \frac{VelikostFS}{Velikostclusteru}$$

3.2 Utils

V souboru `Utils.cpp` je implementována jedna metoda, která kontroluje validnost zadané cesty a vrátí pair ze standardní knihovny C++ s cestou a clusterem. Metoda dokáže zpracovat cesty Absolutní i Relativní. Absolutní cesta začíná znakem `/`, relativní cesta začíná znakem nezačíná `/` a může obsahovat `..` pro návrat o úroveň výše. Pokud je cesta Absolutní metoda pouze projde všechny adresáře a vrátí poslední cluster a cestu. Pokud je cesta Relativní metoda upraví současnou cestu a pak projde všechny adresáře a vrátí poslední cluster a cestu.

4 Příkazy

Všechny příkazy jsou implementovány v separátních souborech `.cpp` ve svém vlastním namespace. Sjednocení všech příkazů je v souboru `Commands.h`.

4.1 Mapování příkazů

K jednoduchému mapování příkazů byla využita Unordered Map ze standardní knihovny C++, které obsahuje klíč (příkaz) a funkci pro předání argumentů příslušným metodám. Tato mapa je velmi jednoduše rozšiřitelná o další příkazy do budoucna.

4.2 Příkaz cp

Implementace příkazu `cp` je v souboru `Cp.cpp`. Nejprve se zkontroluje, zda Souborový Systém existuje a přečte se `Description`. Následně se zkontroluje cesta zdrojového souboru a cílového souboru. Soubory ve zdrojovém adresáři a cílovém adresáři se načtou a zkontrolují se, zda zdrojový i cílový soubor existují. Pokud cílový soubor existuje, kopírování se neprovede. Pokud cílový soubor neexistuje, zkontroluje se, zda je dostatek místa pro kopírování. Následně se zkopírují data ze zdrojového souboru do cílového souboru.

4.3 Příkaz mv

Implementace příkazu `mv` je v souboru `Mv.cpp`. Nejprve se zkontroluje, zda Souborový Systém existuje a přečte se `Description`. Následně se zkontroluje cesta zdrojového souboru a cílového souboru. Soubory ve zdrojovém adresáři a cílovém adresáři se načtou a zkontrolují se, zda zdrojový i cílový soubor existují. Pokud se jména zdroje a cíle neshodují, soubor se přejmenuje. Pokud se cesta zdroje a cíle neshodují, soubor se přesune a předchozí soubor se

smaže (jméno souboru na 0 indexu se nastaví na `\0`). Obsah souboru zůstane v clusterech, kde byl původně uložen.

4.4 Příkaz `rm`

Implementace příkazu `rm` je v souboru `Rm.cpp`. Nejprve se zkontroluje, zda Souborový Systém existuje a přečte se `Description`. Následně se zkontroluje cesta zdrojového souboru. Soubor ve zdrojovém adresáři se načte a zkontroluje se, zda zdrojový soubor existuje. pokud soubor existuje, smaže (jméno souboru na 0 indexu se nastaví na `\0`) se a vymaže obsah clusterů, kde byl uložen.

4.5 Příkaz `mkdir`

Implementace příkazu `mkdir` je v souboru `Mkdir.cpp`. Nejprve se zkontroluje, zda Souborový Systém existuje a přečte se `Description`. Následně se zkontroluje cesta nového adresáře. Adresář se vytvoří a uloží se do adresáře, kam vede cesta.

4.6 Příkaz `rmdir`

Implementace příkazu `rmdir` je v souboru `Rmdir.cpp`. Nejprve se zkontroluje, zda Souborový Systém existuje a přečte se `Description`. Následně se zkontroluje cesta zdrojového adresáře. Adresář se smaže pokud je prázdný.

4.7 Příkaz `ls`

Implementace příkazu `ls` je v souboru `Ls.cpp`. Nejprve se zkontroluje, zda Souborový Systém existuje a přečte se `Description`. Následně se zkontroluje cesta adresáře. Adresář nalezne všechny soubory a adresáře, které obsahuje a vypíše je. Je také možné vypsat obsah aktuálního adresáře.

4.8 Příkaz `cat`

Implementace příkazu `cat` je v souboru `Cat.cpp`. Nejprve se zkontroluje, zda Souborový Systém existuje a přečte se `Description`. Následně se zkontroluje cesta zdrojového souboru. Soubor se načte a vypíše se jeho obsah do konzole.

4.9 Příkaz cd

Implementace příkazu `cd` je v souboru `Cd.cpp`. Nejprve se zkontroluje, zda Souborový Systém existuje a přečte se `Description`. Následně se zkontroluje cesta nového adresáře. Současná cesta se změní na novou cestu.

4.10 Příkaz pwd

Jedná se pouze o vypsaní aktuální cesty.

4.11 Příkaz info

Implementace příkazu `info` je v souboru `Info.cpp`. Nejprve se zkontroluje, zda Souborový Systém existuje a přečte se `Description`. Následně se zkontroluje cesta zdrojového souboru. Soubor se načte a vypíše se pozice souboru v clusterech.

4.12 Příkaz incp

Implementace příkazu `incp` je v souboru `Incp.cpp`. Zkontroluje se, zda vstupní soubor existuje. Nejprve se zkontroluje, zda Souborový Systém existuje a přečte se `Description`. Následně se zkontroluje cesta zdrojového souboru a cílového souboru. Zkontroluje se, zda cílový soubor existuje a zda je dostatek místa pro kopírování. Následně se zkopírují data ze zdrojového souboru a rozdělí se na fragmenty o velikosti clusteru a uloží se do clusterů. Ve FAT se zapíše řetězec clusterů, kde se soubor nachází.

4.13 Příkaz outcp

Implementace příkazu `outcp` je v souboru `Outcp.cpp`. Nejprve se zkontroluje, zda Souborový Systém existuje a přečte se `Description`. Následně se zkontroluje cesta zdrojového souboru a cílového souboru. Zkontroluje se, zda zdrojový soubor existuje a zda cesta do cílového souboru existuje. Následně se zkopírují data z clusterů a uloží se do cílového souboru.

4.14 Příkaz load

Implementace příkazu `load` je v souboru `Load.cpp`. Příkaz simuluje funkci uživatelské konzole. Přečte se soubor, který obsahuje příkazy a postupně se vykonávají. Nezáleží na tom, jaký je typ souboru, pokud je dodržen formát příkazů.

- Pokud je systém nový a nezformátovaný, první provedený příkaz musí být `format <size>MB`.
- Řádky, které začínají znakem `#` jsou ignorovány.
- Každý řádek může obsahovat pouze jeden příkaz.

4.15 Příkaz format

Implementace příkazu `format` je v souboru `Format.cpp`. `Format` vytvoří nebo přepíše souborový systém na požadovanou velikost. Provedou se potřebné výpočty na velikost souborového systému, počet clusterů a a pozice FAT a Data adresy. Jako první se zapíše informace o souborovém systému (`Description`). Následně se vytvoří FAT tabulka začínající na `fat_start_adress` a kořenový adresář začínající na `data_start_adress`. Zbytek souboru se vyplní nulami.

4.16 Příkaz check

Implementace příkazu `check` je v souboru `Check.cpp`. Příkaz kontroluje zda se nachází poškozený cluster v FAT tabulce. Pokud se poškozený cluster nachází, vypíše se jeho pozice clusteru a souborový systém se označí jako poškozený.

4.17 Příkaz bug

Implementace příkazu `bug` je v souboru `Bug.cpp`. Příkaz poškodí souborový systém tak, aby bylo možné ověřit funkčnost příkazu `check`. Nejprve se zkontroluje, zda Souborový Systém existuje a přečte se `Description`. Následně se zkontroluje cesta zdrojového souboru. Soubor se načte a zkontroluje se, zda zdrojový soubor existuje. Na pozici clusteru ve FAT se zapíše hodnota `FAT_BAD_CLUSTER`. Souborový systém se označí jako poškozený.

4.18 Příkaz help

Implementace příkazu `help` je v souboru `Help.cpp`. Příkaz vypíše všechny dostupné příkazy a jejich popis.

4.19 Příkaz exit

Příkaz ukončí program.

5 Uživatelská příručka

Ve složce se kromě této dokumentace nachází soubor `Makefile` pro Unixové systémy a `Makefile.win` pro Windows. Pro sestavení programu je nutné mít nainstalovaný `g++` a `make`.

Unixové systémy:

- `make -f Makefile` – sestaví program a vytvoří spustitelný soubor `File_System`
- `make -f Makefile clean` – smaže soubory vytvořené při sestavení

Windows:

- `make -f Makefile.win` – sestaví program a vytvoří spustitelný soubor `File_System.exe`
- `make -f Makefile.win clean` – smaže soubory vytvořené při sestavení

Program se spouští s jedním argumentem, který reprezentuje souborový systém s příponou `.dat`. Po spuštění programu se zkontroluje stav souborového systému předaného argumentem. Po úspěšné kontrole je možné zadávat příkazy. Všechny funkční příkazy jsou popsány po zadání příkazu `help`. Pro ukončení programu je nutné zadat příkaz `exit`.